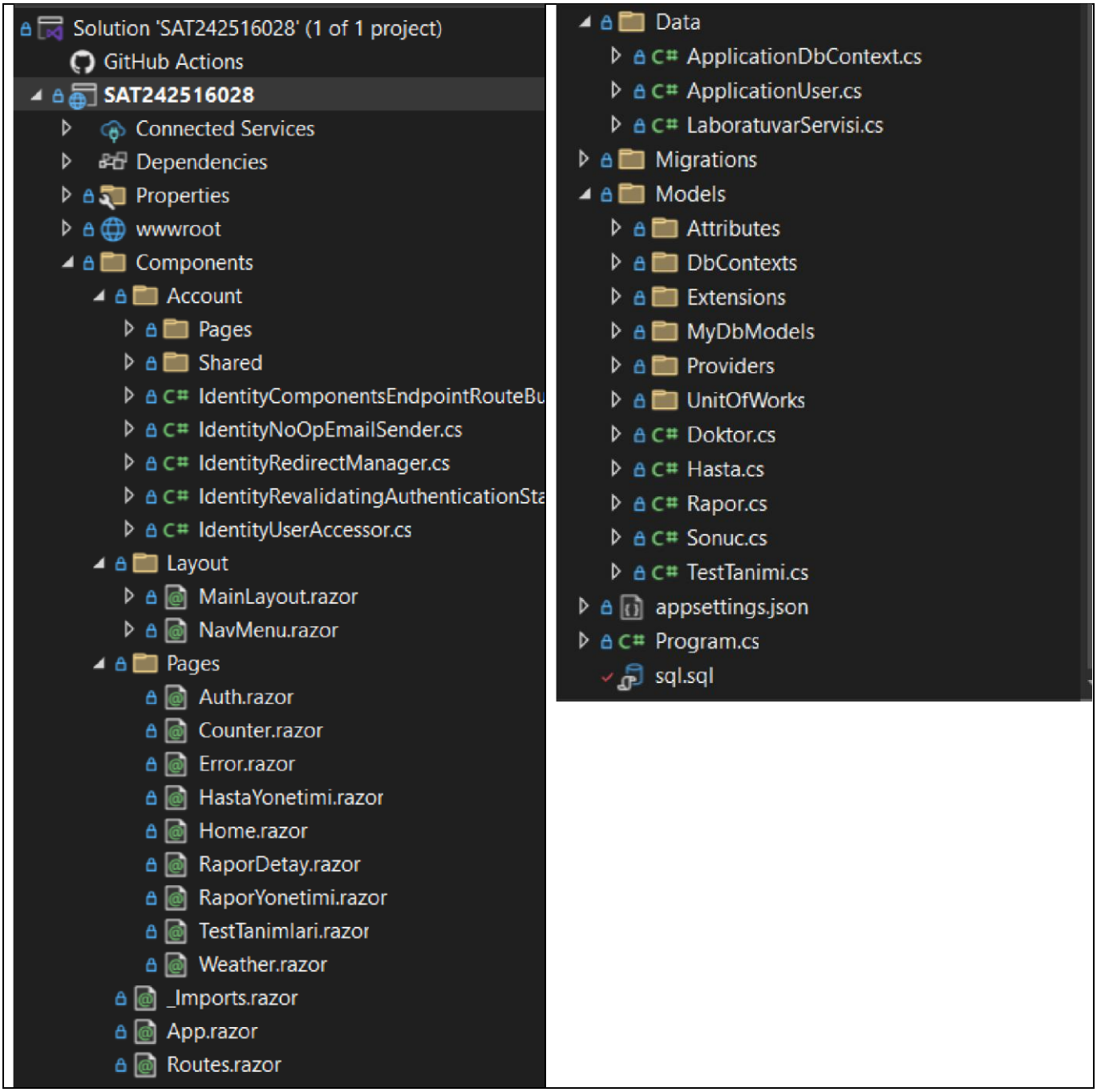
Proje Adı	Laboratuvar Sonuc Yönetimi
Proje Konusu	Laboratuvar sonucları
Proje Çözüm Adı	SAT242516028

Proje 'Çözüm Gezgini (Solution Explorer)' penceresinin (alt klasörler ve dosyalar dahil olacak şekilde) ekran görüntüsünü resim olarak ekleyiniz



Projede oluşturulan arabirimlerin (interface) C# kodlarını yazınız.

```
public interface IMyDbModel
{
    string Message { get; set; }
    IMyDbModel_Parameter Parameters { get; set; }
    IDictionary<object, object> OrderByItems { get; set; }
}
```

```
public interface IMyDbModel<T> : IMyDbModel where T : class, new() {
    IEnumerable<T> Items { get; set; } }
```

```
public interface IMyDbModel_Parameter
{
    string OrderBy { get; set; }
int PageNumber { get; set; }     int
PageSize { get; set; }     int
TotalPageCount { get; }     int
TotalRecordCount { get; set; }
    IDictionary<string, object> Params { get; set; }
    IDictionary<string, string> Where { get; set; } }
```

```
public interface IMyDbModel_Result_KeyValue<TKey,TValue>  {
    TKey Key { get; set; }
    TValue Value { get; set; } }
```

```
public interface IMyDbModel_Provider
{
    ValueTask<IMyDbModel<TResult>> Execute<TResult>(IMyDbModel<TResult>
myResultModel,
        string spName = "",
        bool isPagination = true) where TResult : class, new();
    ValueTask<IMyDbModel<TResult>> Execute<TResult>(string spName = "",
params (string Key, object Value)[] parameters)
        where TResult : class, new();

    ValueTask<IEnumerable<TResult>> GetItems<TResult>(string spName = "",
params (string Key, object Value)[] parameters)
        where TResult : class, new();

    ValueTask<IEnumerable<TResult>> SetItems<TResult>(string spName = "",
params (string Key, object Value)[] parameters)
        where TResult : class, new(); }
```

```
public interface IMyDbModel_UnitOfWork
{
    Task Execute<T>(IMyDbModel<T> myDbModel, string spName = "", bool isPagination =
true)
        where T : class, new(); }
```

Projede oluşturulan sınıfların (class) C# kodlarını yazınız.

Uyarı : Sınıf içerisinde metotların sadece isim ve parametreleri yazılacak.

Örnek : public async Task MethodAsync(parameters…) { return null; }

```
public sealed class MyDbModel<T> : IMyDbModel<T> where T : class, new()
{
    public MyDbModel() : this(1, 10, "")
    {
    }
    public MyDbModel(int pageNumber, int pageSize, string orderBy)
{
        Parameters = MyDbModel_Parameter.Create(pageNumber, pageSize, orderBy);
        OrderByItems = this.GetOrderByItems();
        Items = new List<T>();
    }
    public IMyDbModel_Parameter Parameters { get; set; }
public IDictionary<object, object> OrderByItems { get; set; }
    public IEnumerable<T> Items { get; set; }
public string Message { get; set; } }
```

```
using System;
using System.Collections.Generic;

namespace MyDbModels
{
    public interface IMyDbModel_Parameter
    {
        string OrderBy { get; set; }
int PageNumber { get; set; }         int
PageSize { get; set; }         int
TotalPageCount { get; }         int
TotalRecordCount { get; set; }
        IDictionary<string, object> Params { get; set; }
        IDictionary<string, string> Where { get; set; }
    }
    internal sealed class MyDbModel_Parameter : IMyDbModel_Parameter
    {
        public static MyDbModel_Parameter Create(int pageNumber, int pageSize, string
orderBy)         {
            return null;
        }

        private MyDbModel_Parameter(int pageNumber, int pageSize, string orderBy)
        {
        }

        public int PageNumber { get; set; }
public int PageSize { get; set; }
        public int TotalRecordCount { get; set; }

        public int TotalPageCount
        {
            get { return 0; }
        }

        public string OrderBy { get; set; }
        public IDictionary<string, object> Params { get; set; }
public IDictionary<string, string> Where { get; set; }
    }
}
```

```
class MyDbModel_Result_KeyValue namespace MyDbModels
{
    public interface IMyDbModel_Result_KeyValue<TKey, TValue>
{
        TKey Key { get; set; }
        TValue Value { get; set; }
    }      public class MyDbModel_Result_KeyValue<TKey,
TValue> :
IMyDbModel_Result_KeyValue<TKey, TValue>
    {
        public TKey Key { get; set; }
public TValue Value { get; set; }
    }
}
```

```
using Extensions;
using System.Collections.Generic;

namespace MyDbModels;

public static class MyDbModel_Extension
{
    public static IDictionary<object, object> GetOrderByItems<E>(this MyDbModel<E>
myDbModel) where E : class, new()
    {
        return null;
    }
}
```

```
using MyDbModels; using
UnitOfWorks;
using System.Collections.Generic; using
System.Threading.Tasks;

namespace Providers
{
    public interface IMyDbModel_Provider
    {
        ValueTask<IMyDbModel<TResult>> Execute<TResult>(IMyDbModel<TResult>
myResultModel,
            string spName = "",
            bool isPagination = true) where TResult : class, new();
        ValueTask<IMyDbModel<TResult>> Execute<TResult>(string spName = "",
params (string Key, object Value)[] parameters)             where TResult
: class, new();

        ValueTask<IEnumerable<TResult>> GetItems<TResult>(string spName = "",
params (string Key, object Value)[] parameters)             where TResult :
class, new();

        ValueTask<IEnumerable<TResult>> SetItems<TResult>(string spName = "",
params (string Key, object Value)[] parameters)             where TResult :
class, new();
    }      public class MyDbModel_Provider(IMyDbModel_UnitOfWork
myDbModel_UnitOfWork) :
IMyDbModel_Provider
    {
        public async ValueTask<IMyDbModel<TResult>>
Execute<TResult>(IMyDbModel<TResult> myResultModel,
string spName = "",
            bool isPagination = true) where TResult : class, new()
        {
            return null;
        }
        public async ValueTask<IMyDbModel<TResult>> Execute<TResult>(string spName =
"",
            params (string Key, object Value)[] parameters)
where TResult : class, new()
        {
            return null;
        }
        public async ValueTask<IEnumerable<TResult>> GetItems<TResult>(string spName
= "",
            params (string Key, object Value)[] parameters)
where TResult : class, new()
        {
            return null;
        }
        public async ValueTask<IEnumerable<TResult>> SetItems<TResult>(string spName
= "",
            params (string Key, object Value)[] parameters)
where TResult : class, new()
        {
            return null;
        }
    }
}
```

```
using System.Data; using
Extensions;
using Microsoft.EntityFrameworkCore; using
MyDbModels;
using System.Threading.Tasks;

namespace UnitOfWorks
{
    public interface IMyDbModel_UnitOfWork
    {
        Task Execute<T>(IMyDbModel<T> myDbModel, string spName = "", bool
isPagination = true)
            where T : class, new();
    }         public  sealed  class  MyDbModel_UnitOfWork<TDbContext>(TDbContext
context) :
IMyDbModel_UnitOfWork where TDbContext : DbContext
    {
        public async Task Execute<T>(IMyDbModel<T>
myDbModel,             string spName = "",             bool
isPagination = true)             where T : class, new()
        {
            await Task.CompletedTask;
        }
    }
}
```

```
namespace DbContexts
{
    using Microsoft.EntityFrameworkCore;
     public class MyDbModel_DbContext(DbContextOptions<MyDbModel_DbContext> options)
:
DbContext(options)
    {
    }
}
```

```
namespace Extensions
{
    using System.Data;     using
System.Reflection;     using
System.Collections.Generic;
using System;     using System.Linq;

    public static class Extensions_DataTable
    {
        public static IEnumerable<T> DataTableToList<T>(this DataTable table) where T
: class
{
            return null;
        }

        public static T GetObject<T>(this DataRow row, List<string>
columnsName) where T : class         {
            return null;
        }
    }
}
```

```
using Attributes; using
System;

namespace Extensions
{
    public static class Extensions_Enum
    {
        public static string Color<T>(this T value)
        {
            return null;
        }

        public static string Title<T>(this T value)
        {
            return null;
        }
    }
}
```

```
namespace Extensions
{
    using System;
    using
System.Collections.Generic;
using System.Text.Encodings.Web;
using System.Text.Json;     using
System.Text.Unicode;

    public static class Extensions_Json
    {
        public static T JsonToItem<T>(this string jsonItem)
        {
            return default(T);
        }

        public static string ItemToJson<T>(this T item)
        {
            return null;
        }

        public static List<T> JsonToList<T>(this string json)
        {
            return null;
        }

        public static string ListToJson<T>(this List<T> list)
        {
            return null;
        }
    }
}
```

```
namespace Extensions
{
    using System.Data;
    using Microsoft.Data.SqlClient;     using
System.Collections.Generic;     using System;

    public static class Extensions_SqlParameter
    {
        public static SqlParameter ToSqlParameter_Table_Type_Dictionary<TKey, TValue>(
            this IDictionary<TKey, TValue> dictionary,
string parameterName,             string parameterTypeName =
"",
            int length = 0,
            SqlDbType sqlDbType = SqlDbType.Structured,
            ParameterDirection direction = ParameterDirection.Input)
        {
            return null;
        }

        public static SqlParameter ToSqlParameter_Data_Type<T>(
            this T value,
            string parameterName,
            ParameterDirection direction = ParameterDirection.Input,
            SqlDbType sqlDbType = SqlDbType.NVarChar)
        {
            return null;
        }
    }
}
```

```
namespace Attributes
{
    using System;

    public class ColorAttribute(string color) : Attribute
    {
        public string Color { get; set; }
    }
}
```

```
namespace Attributes
{
    using System;

    public class ColorAttribute(string color) : Attribute
    {
        public string Color { get; set; }
    }
}
```

appsettings.json dosyasında "DefaultConnection" ifadesini projenize göre yazınız

```
DefaultConnection : Server=localhost,1445; Database=SAT242516028; User Id=sa;
Password=Oo_454545; TrustServerCertificate=True;
```

Program.cs dosyasında, gerekli servis kayıtlarını yapan C# kodlarını yazınız.

```
builder.Services.AddRazorComponents()
.AddInteractiveServerComponents();
builder.Services.AddCascadingAuthenticationState();
builder.Services.AddScoped<IdentityUserAccessor>();
builder.Services.AddScoped<IdentityRedirectManager>();
builder.Services.AddScoped<AuthenticationStateProvider,
IdentityRevalidatingAuthenticationStateProvider>();
builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = IdentityConstants.ApplicationScheme;
options.DefaultSignInScheme = IdentityConstants.ExternalScheme;
})
    .AddIdentityCookies();
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(connectionString));
builder.Services.AddDatabaseDeveloperPageExceptionFilter();
builder.Services.AddIdentityCore<ApplicationUser>(options =>
options.SignIn.RequireConfirmedAccount = true)
.AddEntityFrameworkStores<ApplicationDbContext>()
    .AddSignInManager()     .AddDefaultTokenProviders();
builder.Services.AddSingleton<IEmailSender<ApplicationUser>,
IdentityNoOpEmailSender>();
builder.Services.AddSingleton<IConfiguration>(builder.Configuration);
builder.Services.AddScoped<LaboratuvarServisi>();
```

App.razor bileşeninde, gerekli C# kodlarını yazınız

```
  <Routes />
  <script src="_framework/blazor.web.js"></script>
```

Projenizde oluşturmuş olduğunuz bileşenlerin tasarım ve C# kodlarını yazınız (Blazor Component)

NOT: daha fazla bileşen için, aşağıdaki tabloyu çoğaltınız.

Bileşen Adı	RaporYonetimi

Tasarım Kodları	

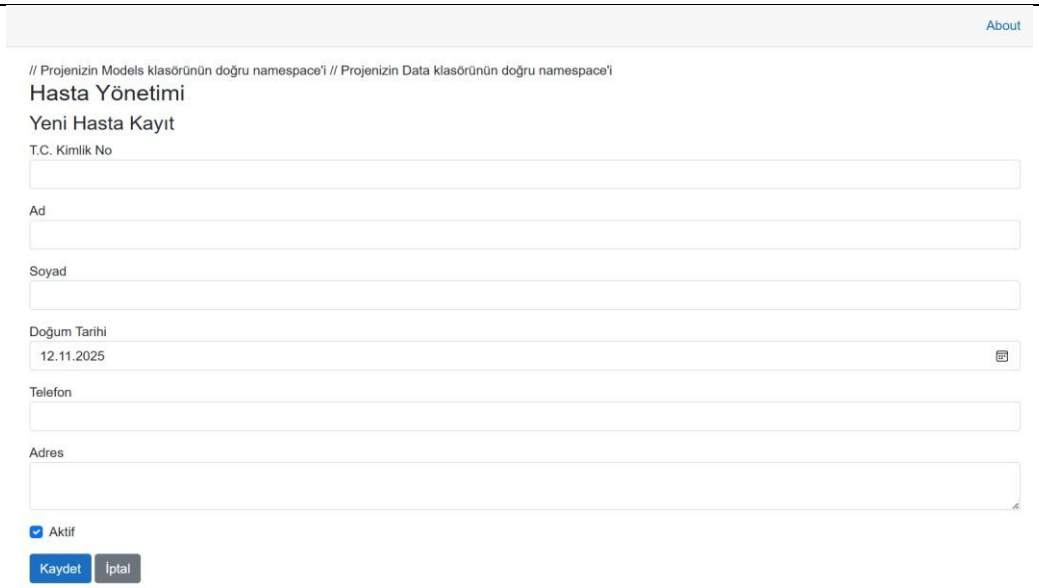
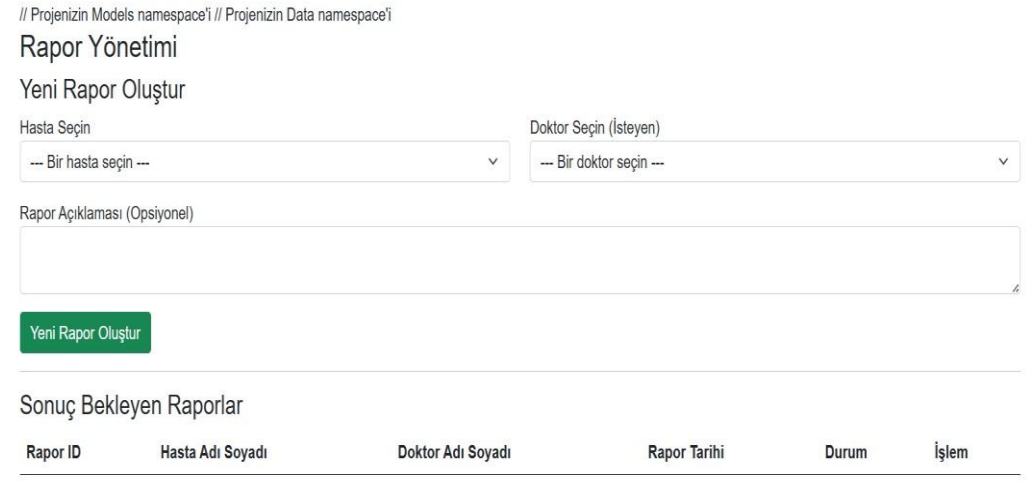
```
@page "/raporlar"
@using SAT242516028.Models // Projenizin Models namespace'i
@using SAT242516028.Data   // Projenizin Data namespace'i
@inject LaboratuvarServisi LabService // Yeni servisimiz
@inject NavigationManager NavigationManager

<h3>Rapor Yönetimi</h3>

<EditForm Model="@yeniRaporFormu" OnValidSubmit="HandleRaporOlustur">
    <DataAnnotationsValidator />

    <h4>Yeni Rapor Oluştur</h4>
    <div class="row">
        <div class="col-md-6 mb-3">
            <label>Hasta Seçin</label>
            <InputSelect class="form-select" @bind-
Value="yeniRaporFormu.HastaId">
                <option value="0">--- Bir hasta seçin ---</option>
                @if (hastaListesi != null)
                {
                    @foreach (var hasta in hastaListesi.Where(h =>
h.Aktif))
                    {
                        <option value="@hasta.Id">@hasta.Ad
@hasta.Soyad (@hasta.TCKimlikNo)</option>
                    }
                }
            </InputSelect>
        </div>
        <div class="col-md-6 mb-3">
            <label>Doktor Seçin (İsteyen)</label>
            <InputSelect class="form-select" @bind-
Value="yeniRaporFormu.DoktorId">
                <option value="0">--- Bir doktor seçin ---</option>
                @if (doktorListesi != null)
                {
                    @foreach (var doktor in doktorListesi)
                    {
                        <option
value="@doktor.Id">@doktor.AdSoyad</option>
                    }
                }
            </InputSelect>
        </div>
    </div>
    <div class="mb-3">
        <label>Rapor Açıklaması (Opsiyonel)</label>
        <InputTextArea class="form-control" @bind-
Value="yeniRaporFormu.Aciklama" />
    </div>

    <button type="submit" class="btn btn-success">Yeni Rapor
Oluştur</button>
</EditForm>

<hr />
```

```
<h4>Sonuç Bekleyen Raporlar</h4>
@if (bekleyenRaporlar == null)
{
    <p><em>Yükleniyor...</em></p>
} else
{
    <table class="table table-hover">
```

```
        <thead>
            <tr>
                <th>Rapor ID</th>
                <th>Hasta Adı Soyadı</th>
                <th>Doktor Adı Soyadı</th>
                <th>Rapor Tarihi</th>
                <th>Durum</th>
                <th>İşlem</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var rapor in bekleyenRaporlar)
            {
                <tr>
                    <td>@rapor.Id</td>
                    <td>@rapor.HastaAdSoyad</td>
                    <td>@rapor.DoktorAdSoyad</td>
                    <td>@rapor.RaporTarihi.ToShortDateString()</td>
                    <td><span class="badge bg-warning
textdark">@rapor.Durum</span></td>
                    <td>
                        <a href="/rapor-detay/@rapor.Id" class="btn btn-
primary btn-sm">
                            Sonuç Gir / Detay
                        </a>
                    </td>
                </tr>
            }
        </tbody>
    </table>
}
```

C# Kodları

```
@code {
    private List<Hasta> hastaListesi;
private List<Doktor> doktorListesi;

    private List<Rapor> bekleyenRaporlar;

    private RaporFormModel yeniRaporFormu = new RaporFormModel();
    // Sayfa yüklendiğinde tüm listeleri doldurur
protected override async Task OnInitializedAsync()
    {
        await ListeleriYukle();
    }

    // Gerekli tüm verileri servisten çeken yardımcı metot
private async Task ListeleriYukle()
    {
        // YENİ METOTLAR:         bekleyenRaporlar = await
LabService.GetBekleyenRaporlarAsync();         hastaListesi = await
LabService.HastaListeleAsync();         doktorListesi = await
LabService.DoktorListeleAsync();     }

    // "Yeni Rapor Oluştur" butonuna basıldığında çalışır
private async Task HandleRaporOlustur()
    {
        if (yeniRaporFormu.HastaId == 0 || yeniRaporFormu.DoktorId ==
0)
        {
            // Basit bir doğrulama
            Console.WriteLine("Hasta veya Doktor seçilmedi.");
return;
        }

        // YENİ METOT:         await
LabService.RaporOlusturAsync(
yeniRaporFormu.HastaId,
yeniRaporFormu.DoktorId,
yeniRaporFormu.Aciklama
        );

        // Formu temizle ve listeyi yenile
yeniRaporFormu = new RaporFormModel();         await
ListeleriYukle();
    }

    // EditForm'a bağlanmak için kullandığımız yardımcı iç sınıf
public class RaporFormModel
    {
        public int HastaId { get; set; }
public int DoktorId { get; set; }
public string Aciklama { get; set; }
    }
}
```

Ekran Görüntüleri

UYARI:
Resim boyutlarının çok büyük olmamasına dikkat ediniz